Orchfile Grammar Reference

Orchfile Specification

This document contains the formal grammar for Orchfile in W3C EBNF notation, followed by railroad (syntax) diagrams for the key production rules. The canonical source is `grammar.ebnf` in the repository root.

Contents

1 EBNF Grammar **2**

1.1 Lines . . . 2
1.2 Document . . . 2
1.3 Service Blocks . . . 2
1.4 Directives . . . 2
1.4.1 Overlay Composition . . . 2
1.4.2 Execution Mode . . . 3
1.4.3 Container-only Directives . . . 3
1.4.4 Host-only Directives . . . 3
1.4.5 Common Directives . . . 3
1.5 Terminals . . . 4

2 Railroad Diagrams **6**

3 Constraints **8**

1 EBNF Grammar

1.1 Lines

```
/* Lines. Directives start at column 0. Trailing whitespace on a line is not part of
    any value. */
orchfile    ::= bom? line ( newline line )*   /* the final line ending is optional */
line        ::= blank | comment | statement ws?
blank       ::= ws*
comment     ::= ws* '#' any*
bom         ::= #xFEFF   /* optional UTF-8 byte-order mark; ignored */
statement   ::= version | arg | service_decl | directive
```

1.2 Document

```
/* Document. The statements of a file, in order, with blank and comment
   lines removed, MUST form a document. */
document    ::= version? arg* service_block*
version     ::= 'ORCH_VERSION' ws version_number
arg         ::= 'ARG' ws identifier '=' value
```

1.3 Service Blocks

```
/* Service blocks. A block sets at most one mode. A block with neither FROM
   nor RUN is an overlay block and takes its mode from another file (C1). */
service_block  ::= service_decl ( container_body | host_body | overlay_body )
service_decl   ::= 'SERVICE' ws service_name
container_body ::= container_item* from_directive container_item*
host_body      ::= host_item* run_directive host_item*
overlay_body   ::= overlay_item*
container_item ::= container_directive | common_directive | clear_directive | arg
host_item      ::= host_directive | common_directive | clear_directive | arg
overlay_item   ::= container_directive | host_directive | common_directive
                 | clear_directive | arg
```

1.4 Directives

```
directive   ::= from_directive
              | run_directive
              | container_directive
              | host_directive
              | common_directive
              | clear_directive
```

1.4.1 Overlay Composition

```
/* Overlay composition - resets a list-type directive before merge */
clear_directive
            ::= 'CLEAR' ws clearable_target

clearable_target
            ::= 'ENV' | 'ENV_FILE' | 'PUBLISH' | 'VOLUME'
              | 'REQUIRES' | 'AFTER'
```

1.4.2 Execution Mode

```
/* Execution mode */
from_directive
            ::= 'FROM' ws image_ref
run_directive
            ::= 'RUN' ws command_string
```

1.4.3 Container-only Directives

```
/* Container-only */
container_directive
            ::= 'ENTRYPOINT' ws command_string
              | 'CMD' ws command_string
              | 'PUBLISH' ws ( host_address ':' )? port ':' port
              | 'VOLUME' ws volume_source ':' volume_destination ( ':' volume_mode )?
```

1.4.4 Host-only Directives

```
/* Host-only */
host_directive
            ::= 'USER' ws user_name
              | 'STOP' ws command_string
              | 'RELOAD' ws command_string
```

1.4.5 Common Directives

```
/* Common - environment */
env_directive
            ::= 'WORKDIR' ws path
              | 'ENV' ws env_key '=' value
              | 'ENV_FILE' ws path

/* Common - dependencies */
dependency_directive
            ::= 'REQUIRES' ws service_name ( ws service_name )*
              | 'AFTER' ws service_name ( ws service_name )*

/* Common - health */
health_directive
            ::= 'HEALTHCHECK' ws ( url | command_string )
              | 'READINESS_TIMEOUT' ws duration

/* Common - lifecycle */
lifecycle_directive
            ::= 'ONESHOT' ws bool
              | 'DISABLED' ws bool
              | 'RECREATE' ws recreate_policy

/* Common - restart */
restart_directive
            ::= 'RESTART' ws restart_policy
              | 'RESTART_DELAY' ws duration
              | 'START_LIMIT_BURST' ws integer
              | 'START_LIMIT_INTERVAL' ws duration
```

```
/* Common - timeouts */
timeout_directive
            ::= 'TIMEOUT_START' ws duration
              | 'TIMEOUT_STOP' ws duration

/* Common - resources */
resource_directive
            ::= 'MEMORY' ws memory_size
              | 'CPUS' ws number
              | 'CPU_QUOTA' ws percentage
              | 'LIMIT_NOFILE' ws integer
              | 'LIMIT_NPROC' ws integer
              | 'TASKS_MAX' ws integer
              | 'IO_WEIGHT' ws integer   /* 10-1000 */

/* Common - logging */
logging_directive
            ::= 'STDOUT' ws path
              | 'STDERR' ws path

common_directive
            ::= env_directive
              | dependency_directive
              | health_directive
              | lifecycle_directive
              | restart_directive
              | timeout_directive
              | resource_directive
              | logging_directive
```

1.5 Terminals

```
/* Terminals */
service_name ::= letter ( letter | digit | '-' )*   /* max 63 chars */
identifier   ::= ( alpha | '_' ) ( alpha | digit | '_' )*   /* ARG names */
env_key      ::= identifier
user_name    ::= ( alpha | digit | '_' ) ( alpha | digit | '_' | '-' | '.' )*

volume_source      ::= volume_part   /* host path or named volume */
volume_destination ::= volume_part   /* container path */
volume_part ::= ( visible - ':' ) ( ( any - ':' )* ( visible - ':' ) )?
volume_mode ::= 'ro' | 'rw'          /* rw if omitted */

/* Free text. It never starts or ends with whitespace. */
text         ::= visible ( any* visible )?
image_ref    ::= text
command_string ::= text
path         ::= text
value        ::= ( any* visible )?   /* after '='; may be empty */
url          ::= ( 'http://' | 'https://' ) text

/* var_ref and var_escape may appear anywhere in the value of a directive inside a
    SERVICE block. Each value must match its rule after expansion (see SPEC.md). */
var_ref      ::= '${' identifier '}'   /* identifier MUST be a declared ARG */
var_escape   ::= '$${'                 /* a literal '${'; never starts a var_ref */

bool         ::= 'true' | 'false'
```

```
restart_policy ::= 'no' | 'always' | 'on-failure'
recreate_policy ::= 'always' | 'never'

duration    ::= integer ( 's' | 'm' )
memory_size ::= integer ( 'K' | 'M' | 'G' )
percentage  ::= integer '%'
number      ::= integer ( '.' digit+ )?
integer     ::= digit+
port        ::= digit+   /* 1-65535 */
version_number ::= integer '.' integer '.' integer ( '-' prerelease )?
prerelease  ::= ( letter | digit | '-' | '.' )+   /* e.g. rc, beta.1 */

host_address ::= ipv4 | ipv6 | hostname
ipv4        ::= octet '.' octet '.' octet '.' octet
octet       ::= digit+                          /* 0-255 */
ipv6        ::= ( hex_digit | ':' | '.' )+      /* RFC 4291 text form, without
    brackets */
hostname    ::= label ( '.' label )*            /* RFC 1123 */
label       ::= alnum ( ( alnum | '-' )* alnum )?   /* max 63 chars */
alnum       ::= alpha | digit
hex_digit   ::= digit | [a-fA-F]

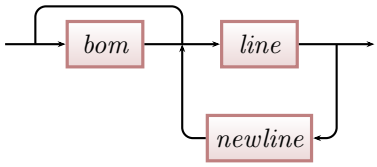
letter      ::= [a-z]
alpha       ::= [a-zA-Z]
digit       ::= [0-9]
ws          ::= [ #x9]+
newline     ::= #xA | #xD #xA
any         ::= #x9 | [#x20-#x7E] | [#xA0-#xD7FF] | [#xE000-#x10FFFF]   /* no control
    characters except tab; no surrogates */
visible     ::= any - [ #x9]
```

Note: The grammar describes each value *after* variable expansion. `var_ref` (`${name}`) may appear anywhere in the value of a directive inside a SERVICE block, and `var_escape` (`$${`) stands for a literal `${`. With `ARG port=8080`, `PUBLISH ${port}:80` is checked as `PUBLISH 8080:80`. Every name must be a declared ARG (C5).

2 Railroad Diagrams

The following diagrams visualize the key production rules. Rounded boxes are *terminals* (literal text), rectangular boxes are *non-terminals* (references to other rules).

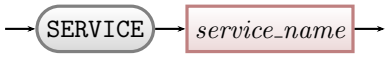
orchfile



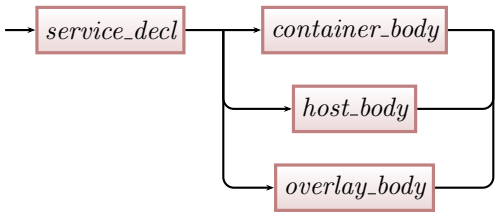
arg



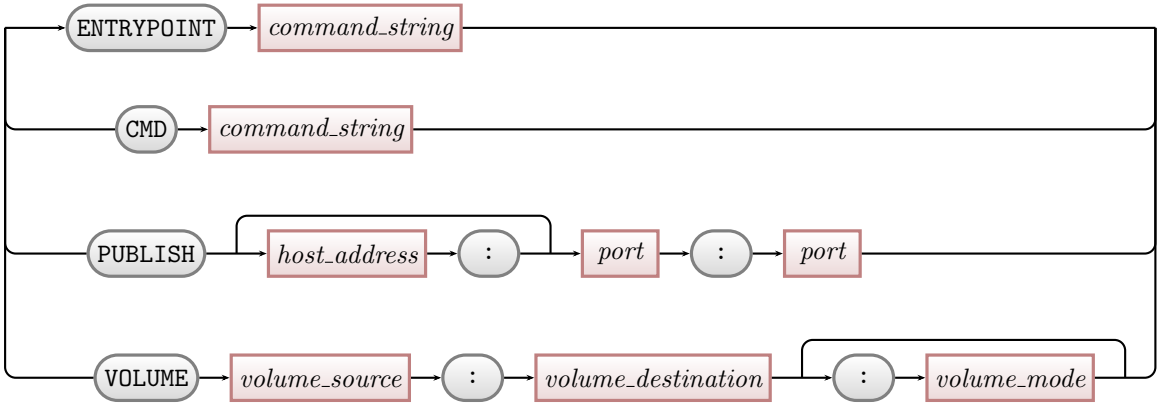
service_decl



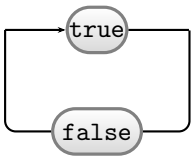
service_block



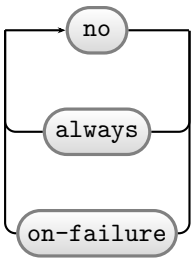
container_directive



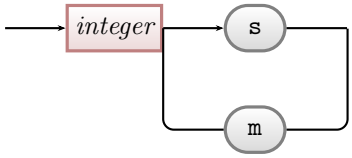
bool



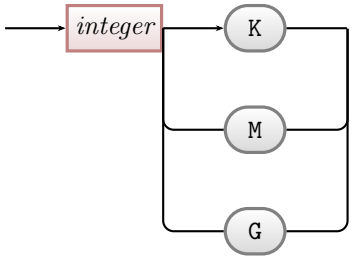
restart_policy



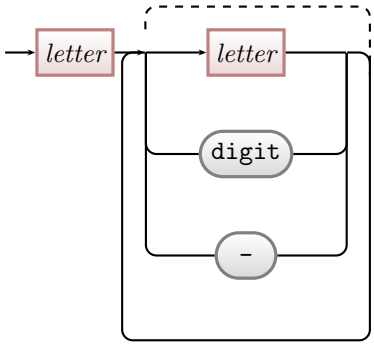
duration



memory_size



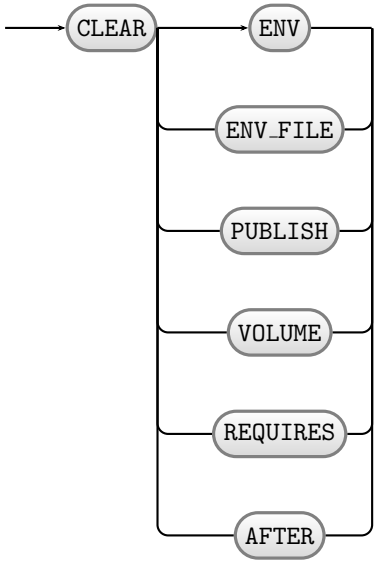
service_name (max 63 characters)



var_ref



clear_directive



3 Constraints

The grammar fixes the shape of each file: a service block sets at most one of `FROM` or `RUN`, and container-only or host-only directives appear only in a block of the matching mode or in an overlay block. The constraints below need information a single file's syntax does not carry: other files, declared names, the dependency graph, or whether a value repeats.

C1: Every service has a mode After all files are merged, some block for each service sets `FROM` (container mode) or `RUN` (host mode).

C2: Container-only directives `ENTRYPOINT`, `CMD`, `PUBLISH` and `VOLUME` in an overlay block require the merged service to be in container mode.

C3: Host-only directives `USER`, `STOP` and `RELOAD` in an overlay block require the merged service to be in host mode.

C4: Dependency acyclicity `REQUIRES` and `AFTER` edges must form a directed acyclic graph (DAG).

C5: Declared variables Every `${name}` must name an `ARG` declared in one of the merged files.

C6: No repeats within a file Within one file, a service sets each scalar directive, each `ENV` key and each `VOLUME` destination at most once. Across files, overlays replace values.

Numeric ranges in the grammar's comments (ports 1–65535, IPv4 octets 0–255, hostname labels and service names up to 63 characters, `IO_WEIGHT` 10–1000) are part of the grammar; they are written as comments because spelling them out in EBNF adds length, not information.